

Full-Stack & SaaS Product Architectural Audit

SkillSetu AI: Comprehensive Codebase Review, User Personas, Feature Inventory & Commercial SaaS Roadmap

Target Codebase: SkillSetu AI (Monorepo)	Date of Audit: September 2026	Status: Hackathon Prototype → Commercial SaaS
Primary Tech Stack: Next.js 15, FastAPI, SQLite, ChromaDB	Author / Role: Full-Stack & SaaS Product Architect	Classification: Enterprise Confidential

1. Executive Summary & Architectural Overview

SkillSetu AI was engineered during a hackathon to solve a pervasive structural bottleneck in technical recruitment: **the unverified resume problem**. Traditional ATS filters blindly reject qualified candidates based on missing keywords, while candidates fabricate skills with generative AI. SkillSetu bridges this gap by introducing a *Weighted Deficit Resistance Model*, modular skill assessments, 15-minute micro-sprints, and tamper-proof SHA-256 micro-credentials.

The Dual Nature of the Current Codebase:

- **Frontend Tier (Next.js 15 App Router + React 19 + TypeScript + Vanilla CSS):** World-class UI/UX with 17 prerendered routes, zero ESLint warnings, dark mode, rich interactive modals, and real-time state management.
 - **Backend Tier (FastAPI + Python 3.11 + SQLite + ChromaDB):** Split between highly functional, data-backed engines (e.g., modular courses, exam grading, GitHub secret scanning, deadline cron) and **simulated hackathon demo mocks** (e.g., regex-based sandbox code execution, static Latin-1 PDF resume parsing, in-memory sprint storage, and hardcoded AI interview scores).
- To commercialize SkillSetu into a paying B2B/B2C SaaS platform, these simulated elements must be replaced with robust, secure, and horizontally scalable cloud services.

2. Comprehensive User Personas & Access Profiles

Persona 1: Student / Candidate / Job Seeker (role === 'student')

- Target Users:** Engineering college students, bootcamp graduates, and junior-to-mid career developers.
- Core Motivations:** Bypassing black-box ATS rejection, proving practical competency through verifiable proof-of-work, and receiving actionable guidance to bridge specific skill gaps.
- Accessible Workflows:** Student Dashboard (/student), Profile & Privacy settings (/student/profile), Secret Shield & Remediation (/student/github-security), 13 Modular Courses & Timed Exams (/hub), Interactive Assessments, ATS Resume Manager, Dynamic Sandbox, and Public Hash Verification (/verify/[hash]).

Persona 2: Employer / Hiring Manager / Recruiter (role === 'employer')

- Target Users:** Technical recruiters, hiring managers, talent acquisition leads, and staffing agencies.
- Core Motivations:** Eliminating resume spam, discovering 'hidden gem' candidates who lack pedigree but have verified execution skills, cutting hiring cycle times from 45 days to 72 hours, and mitigating hiring bias.
- Accessible Workflows:** Employer Portal (/employer), Talent Radar (3-tier candidate segmentation), Blind DEI Screening mode (anonymizing photos, names, and colleges), 1-Click Gap Sprint Dispatcher, 5-stage Pipeline Kanban (Applied → Screened → Shortlisted → Interview → Offer), Job Requisition Creator Modal, and Candidate Deep-Dive Drawer.

Persona 3: Institutional TPO / Academic Admin (role === 'admin')

Target Users: University Training & Placement Officers (TPOs), College Deans, Bootcamp Directors, and Platform Super-Admins.

Core Motivations: Increasing campus placement percentages, achieving NAAC Criterion 5 compliance, identifying systemic batch skill deficits, and ensuring proctoring integrity.

Accessible Workflows: Institutional Admin Console (/admin), Multi-Organization Switcher (Acme, Alpha Staffing, Apex Institute), Plan & Seat Allocation Management, Skill Taxonomy Tree (\$3.0times\$ critical vs \$1.0times\$ optional weights), Anti-Cheat Anomaly Monitor (tab-blur events, \$<45\$s rapid completions), Credential Ledger, and University Cohort Sprint Trigger (batch dispatch to 25+ campus students).

Persona 4: Public Evaluator / Unauthenticated Visitor (Guest)

Target Users: Prospective B2B customers, external HR verification officers, investors, and prospective job seekers.

Core Motivations: Evaluating product features, calculating hiring ROI, viewing enterprise pricing tiers, and verifying candidate credential integrity without account creation.

Accessible Workflows: Public Landing Page (/), B2B/B2C SaaS Pricing & Interactive ROI Calculator (/pricing), 6-Slide Executive Pitch Deck mode, Login / Register onboarding (/login, /register), and Public Cryptographic Certificate Verifier (/verify/[hash]).

3. Complete Feature Inventory & Access Control Matrix

Below is an exhaustive inventory of all 27 distinct features identified across the codebase, mapped against user persona permissions, frontend routes, and backend API endpoints:

#	Feature Module	Capabilities & Scope	Stu	Em p	Ad m	Pu b	Frontend Route / File	Backend Router
1	Auth & Onboarding	Email/pwd login, Role switcher, Student registration	Y	Y	Y	Y	/login, /register	auth.py
2	Pricing & ROI	B2C Pro (Rs 799), B2B Engine (Rs 24,999), ROI calc, Deck	V	V	V	Y	/pricing	Client UI only
3	ATS Resume Parsing	PDF/Docx upload, text extraction, work history & skills	Y	-	-	-	ResumeUploadDrawer.tsx	resume.py, ats.py
4	Deficit Scoring Model	Weighted match: 3.0x critical, 1.0x optional skills	Y	Y	-	-	SkillGapRadar.tsx, TalentRadar	ats_scorer.py
5	Course Catalog (13)	13 modular curricula across full stack, python, aws, data	Y	-	-	-	BackendEngineTabs.tsx	courses.py
6	20-Question Exams	Timed proctored multiple choice exams per course slug	Y	-	-	-	BackendEngineTabs.tsx	courses.py:L90
7	Interactive Assessments	20 production questions, Gold/Silver/Bronze badges (70%)	Y	-	-	-	InteractiveAssessmentModal.tsx	assessments.py
8	Proctored Bug Sandbox	Code editor, terminal logs, challenge gen & test runner	Y	-	-	-	DynamicSandboxModal.tsx	sandbox.py
9	15-min Micro-Sprints	Targeted mini-assessments to bridge single skill gaps	Y	-	-	-	MicroSprintModal.tsx	sprints.py
10	SHA-256 Hash Minting	Deterministic JSON canonicalization & cryptographic hash	Y	-	-	-	ProofOfWorkModal.tsx	crypto.ts, sprints.py
11	Public Cert Verifier	Zero-login cryptographic certificate verification portal	Y	Y	Y	Y	/verify/[hash]	WebCrypto API
12	GitHub Secret Shield	Regex scan for leaked OpenAI, AWS, PAT keys; 1-click git fix	Y	-	-	-	/student/github-security	github.py, features/
13	GitHub AST Audit	Repository complexity analyzer & tech stack identification	Y	-	-	-	GitHubAnalysisModal.tsx	github.ts, career_compass
14	90-Day Career Compass	3-phase personalized career curriculum for target roles	Y	-	-	-	ReadinessRoadmap.tsx	career_roadmap.py
15	AI Interview Coach	Role-specific questions, transcript evaluation & scoring	Y	-	-	-	InterviewCoachModal.tsx	interview_coach.py
16	Job Alerts & Cron	Job feed matching candidate skills, 21d/7d deadline worker	Y	-	-	-	RealtimeJobAlerts.tsx	jobs.py, notifications.py
17	Talent Radar Matrix	Candidate segmentation: Job-Ready (>=85%), Bridgeable, Mismatch	-	Y	-	-	/employer, TalentRadar.tsx	database.py:L278

18	Blind DEI Screening	Masks avatar, name, and college to eliminate hiring bias	-	Y	-	-	TalentRadar.tsx:L125	Frontend State
19	1-Click Sprint Dispatch	Employers dispatch targeted challenges to bridgeable talent	-	Y	-	-	TalentRadar.tsx:L210	sprints.py:L33
20	5-Stage Pipeline Kanban	Applicant tracking: Applied -> Screened -> Shortlist -> Offer	-	Y	-	-	/employer:L250	Frontend Store
21	Candidate Drawer	Detailed profile: verified skills, sprint completions, hashes	-	Y	-	-	CandidateDrawer.tsx	database.py
22	Job Creator Modal	Create job requisitions with salary, weights & thresholds	-	Y	-	-	/employer:L180	jobs.py
23	Multi-Org & Seats	Switch between Acme, Alpha Staffing; monitor seat limits	-	-	Y	-	/admin:L85	Frontend Store
24	Skill Taxonomy Engine	Configure taxonomy tree; toggle Critical (3x) vs Optional (1x)	-	-	Y	-	/admin:L140	database.py:L37
25	Anti-Cheat Monitor	Track tab-blur switching & <45s rapid completion anomalies	-	-	Y	-	/admin:L210	Frontend Store
26	Credential Ledger	Global tamper-proof audit trail of all minted credentials	-	-	Y	-	/admin:L270	Frontend Store
27	Cohort Sprint Trigger	Batch dispatch remediation challenges to 25+ campus students	-	-	Y	-	/admin:L320	campusCandidatesSeed

4. The 'Reality Check': Real Code vs. Hackathon Demo Mocks

A critical finding of this audit is that multiple modules operate as **heuristic simulations** designed to impress hackathon judges, rather than real execution pipelines. While architecturally brilliant for a 48-hour presentation, they will fail immediately in commercial production.

4.1 Proctored Bug-Fix Sandbox (DynamicSandboxModal.tsx & sandbox.py)

The Claim: Containerized cloud execution runner that runs candidate code against unit tests, validates execution plans, and monitors P99 latency dropping from 1,420ms to 11ms.

The Code Evidence: backend/app/api/v1/sandbox.py lines 171-223.

if 'create index' in code_lower and ('on ' in code_lower or 'users' in code_lower):

```
    is_solved = True
```

```
    tests = [TestCaseResult(name='Execution Plan EXPLAIN (ANALYZE)', status='PASS', latency_metric='1,420ms -> 11ms (-99.2%)')]
```

```
elif len(code_clean) > 25 and not any(k in code_clean for k in ['TODO', 'pass']):
```

```
    is_solved = True
```

Commercial SaaS Reality: There is **NO Docker container**, **NO gVisor sandbox**, **NO subprocess isolation**, and **NO database execution**. Any submission containing the substring create index or any 26-character string is flagged as solved with hardcoded string templates.

4.2 ATS Resume Parsing Drawer (ResumeUploadDrawer.tsx & resume.py)

The Claim: Computer-vision spatial layout analyzer and LLM entity extractor parsing any candidate PDF into structured JSON.

The Code Evidence: backend/app/api/v1/resume.py lines 73-125.

```
def mock_ai_parse_pdf_content(file_bytes: bytes, filename: str):
```

```
    text_content = file_bytes.decode('latin-1', errors='ignore')
```

```
    is_fresher = ('intern' in text_content.lower() or 'student' in text_content.lower())
```

```
    return {'personal_info': {'full_name': 'Aditya Verma', 'email': 'aditya.verma@example.com'}, ...}
```

Commercial SaaS Reality: Regardless of what PDF is uploaded to /api/v1/resume/upload-pdf, it decodes bytes with Latin-1 and returns a static JSON payload for 'Aditya Verma' with experience at 'Nexus Scale Labs'.

4.3 AI Interview Coach (InterviewCoachModal.tsx & interview_coach.py)

The Claim: Generates adaptive technical questions and scores voice/text answers across clarity, technical accuracy, and confidence.

The Code Evidence: backend/features/interview_coach.py line 8 & backend/app/api/v1/career_compass.py lines 130-139.

features/interview_coach.py explicitly states:

'The actual AI/API integration will be handled separately.'

career_compass.py line 131:

eval_result = build_interview_evaluation(overall_score=88.0, clarity=90.0, technical_accuracy=86.0, confidence_estimate=0.92, ...)

Commercial SaaS Reality: The backend returns the identical hardcoded question q-dist-01 and the identical score of **88.0%** to every candidate regardless of what answer they submit.

4.4 90-Day Career Compass Roadmap (ReadinessRoadmap.tsx & career_roadmap.py)

The Claim: Generates a personalized 90-day learning curriculum tailored to the candidate's target job role and skill deficiencies.

The Code Evidence: backend/features/career_roadmap.py line 8.

'API/LLM integration is intentionally kept separate.'

def prepare_career_roadmap(...):

return {'roadmap_requirements': {'duration_days': 90, ...}}

Commercial SaaS Reality: The endpoint merely echoes back a validation dictionary. The frontend falls back to a static hardcoded 3-phase curriculum.

4.5 Gap Sprint In-Memory Persistence (sprints.py)

The Code Evidence: backend/app/api/v1/sprints.py line 18.

_DISPATCHED_SPRINTS: Dict[str, Dict[str, Any]] = {}

Commercial SaaS Reality: All dispatched sprint challenges are stored in a transient Python dictionary. If the FastAPI process restarts or scales to multiple workers, all active sprint tokens and URLs are permanently lost.

4.6 Client-Side Authentication & API Token Omission (RoleGuard.tsx & client.ts)

The Code Evidence: frontend/src/app/login/page.tsx line 97 & frontend/src/lib/api/client.ts line 31.

In login/page.tsx, regardless of whether the backend API call succeeds or fails, the finally block executes login(role, email) in React context. In client.ts, the fetch wrapper **never attaches the JWT token from localStorage to outgoing requests as an Authorization: Bearer header**.

Commercial SaaS Reality: All backend endpoints are public and unauthenticated. Any user can switch to Admin or Employer role by altering React state in the browser developer tools.

4.7 SaaS Billing & Subscriptions (pricing/page.tsx)

The Code Evidence: frontend/src/app/pricing/page.tsx line 320.

Clicking 'Select Plan' opens a client-side modal (selectedPlanModal) displaying a simulation prompt. There is no Stripe or Razorpay webhook, payment intent, or checkout session.

5. Genuine, Production-Functional Backend Engines

Alongside the hackathon shortcuts, the audit identified **6 core modules that are genuinely functional, data-backed, and production-ready**:

1. Modular Course Catalog & Timed Mock Test Engine (courses.py & /courses)

Reads 13 actual JSON-authored course curricula from disk (course.json, lessons.json, mock_test.json). The mock exam evaluation endpoint (courses.py:L90) performs genuine grading against answer keys, computes percentages, assigns pass/fail status, and mints real SHA-256 credentials.

2. Interactive Skill Assessments (assessments.py)

Contains a curated repository of 20 production questions across PostgreSQL, Python, Docker, FastAPI, and React. Genuinely writes attempts to the SQLite assessments table, updates candidate readiness scores, and awards badge tiers based on a 70% threshold.

3. GitHub Secret Scanner & 1-Click Remediation (github.py & features/github_analysis.py)

Connects to <https://api.github.com> via httpx, scans public repository files for leaked OpenAI, AWS, and PAT keys using regex patterns, and pushes real commits to candidate repos to add .gitignore or purge .env files when provided with a PAT.

4. Real-time Job Alerts & Deadline Cron Worker (jobs.py, notifications.py, notification_worker.py)

Backed by SQLite jobs and user_notifications tables populated with 60 realistic Indian tech jobs. The cron worker computes real calendar day differences (21-day and 7-day deadlines) and writes actionable notification alerts.

5. Deterministic Cryptographic Proof-of-Work Minting (crypto.ts & /verify/[hash])

Uses the native Web Crypto API (window.crypto.subtle.digest) to canonicalize JSON payloads and compute true SHA-256 hashes. The public verification route dynamically validates hash integrity with zero login required.

6. Talent Radar & Blind DEI Screening Engine (TalentRadar.tsx & ats_scorer.py)

Computes candidate tier placements using the weighted deficit resistance formula ($\$3.0 \times \$ \text{critical}$, $\$1.0 \times \$ \text{optional}$). The blind screening toggle replaces avatars, names, and tier-colleges with synthetic tokens in real-time.

6. Commercial SaaS Transformation Roadmap

To commercialize SkillSetu into an enterprise-grade SaaS platform, development should follow a phased 4-milestone engineering plan:

Milestone 1: Production Multi-Tenancy & Hardened RBAC (Sprint 1-2)

- **Database Migration:** Migrate from local SQLite (skillsetu.db) to managed PostgreSQL (Supabase / Neon / AWS RDS).
- **Relational Schema:** Implement tables for organizations, organization_memberships, subscriptions, dispatched_sprints, and verified_credentials using SQLAlchemy 2.0 & Alembic migrations.
- **JWT Authentication:** Integrate Clerk or Auth0. Update client.ts to attach Bearer tokens to all requests, and enforce FastAPI dependencies with role checks (require_role('employer')).

Milestone 2: Real Containerized Code Execution Sandbox (Sprint 3-4)

- **Isolated Code Runner:** Replace regex heuristics with an isolated container backend (**Judge0 API** or ephemeral Docker containers via Docker SDK).
- **Database Performance Sandbox:** Spin up ephemeral PostgreSQL instances populated with 50,000 rows. Execute candidate DDL/SQL queries and extract authentic EXPLAIN (ANALYZE, BUFFERS) execution metrics.
- **Async Python Sandbox:** Execute candidate code in an isolated subprocess running pytest with resource limits (1 CPU, 256MB RAM, 5-second execution timeout).

Milestone 3: Real LLM Orchestration & Evaluation (Sprint 5-6)

- **Unified ATS Parser:** Deprecate the mock in resume.py and consolidate on ats.py using pdfplumber + Groq/Gemini structured outputs (instructor library).
- **Authentic AI Interview Coach:** Connect career_compass.py to an LLM evaluator that analyzes candidate audio/text transcripts against question rubrics, outputting dynamic scores and personalized improvements.
- **Dynamic Career Roadmaps:** Connect career_roadmap.py to the course catalog database to construct genuine personalized learning paths.

Milestone 4: SaaS Billing, Metering & Webhooks (Sprint 7-8)

- **Payment Gateways:** Integrate **Razorpay** (for India: UPI, NetBanking) and **Stripe** (for global B2B clients).
- **Subscription Webhooks:** Build FastAPI endpoints to process checkout.session.completed, invoice.paid, and subscription.deleted events.
- **Usage Metering:** Implement tenant limits (Starter: 3 active jobs, 50 candidate screens/mo; Growth: 10 active jobs, 300 screens/mo; Enterprise: custom).

7. Executive Readiness Scorecard

Architectural Dimension	Hackathon Score	Status	Commercial SaaS Target Action
Frontend UI / UX	9.5 / 10	Exceptional	Retain current design system; wire authentic API states and error boundaries.
Domain Logic & Scoring	8.5 / 10	Strong	Retain Deficit Resistance formula; enforce via PostgreSQL constraints.
Database & Multi-Tenancy	4.0 / 10	Prototype	Migrate SQLite & in-memory dicts to PostgreSQL + Row Level Security.
Code Sandbox Execution	2.0 / 10	Mock	Replace substring regex matching with Judge0 / Docker container execution.
AI & LLM Integration	3.5 / 10	Simulated	Wire structured LLM pipelines with schemas, streaming, and validation.
Monetization & Billing	1.0 / 10	UI Only	Integrate Stripe & Razorpay webhooks, seat limits, and usage metering.

8. Strategic Next Steps for SaaS Execution

- Immediate Priority Order:**
- Sprint 1:** Replace SQLite and in-memory sprint state with managed PostgreSQL and Alembic migrations. Implement JWT Bearer token authentication in client.ts with FastAPI role dependency verification.
 - Sprint 2:** Implement a real isolated code runner using Judge0 API or a containerized Docker runner to replace regex string matching in sandbox.py.
 - Sprint 3:** Consolidate PDF resume parsing onto a single production pipeline using pdfplumber and Gemini/Groq structured outputs.
 - Sprint 4:** Integrate Razorpay (India) and Stripe (Global) checkout webhooks to activate the B2B and B2C pricing tiers.

Audit Reviewer: Lead Full-Stack & SaaS Product Architect	Target Codebase: SkillSetu AI Monorepo	Review Status: Complete & Approved
Architecture Rating: High Potential (Pre-SaaS)	Next Milestone: Phase 5 PostgreSQL & Sandbox Migration	Distribution: Engineering Leadership & Founders